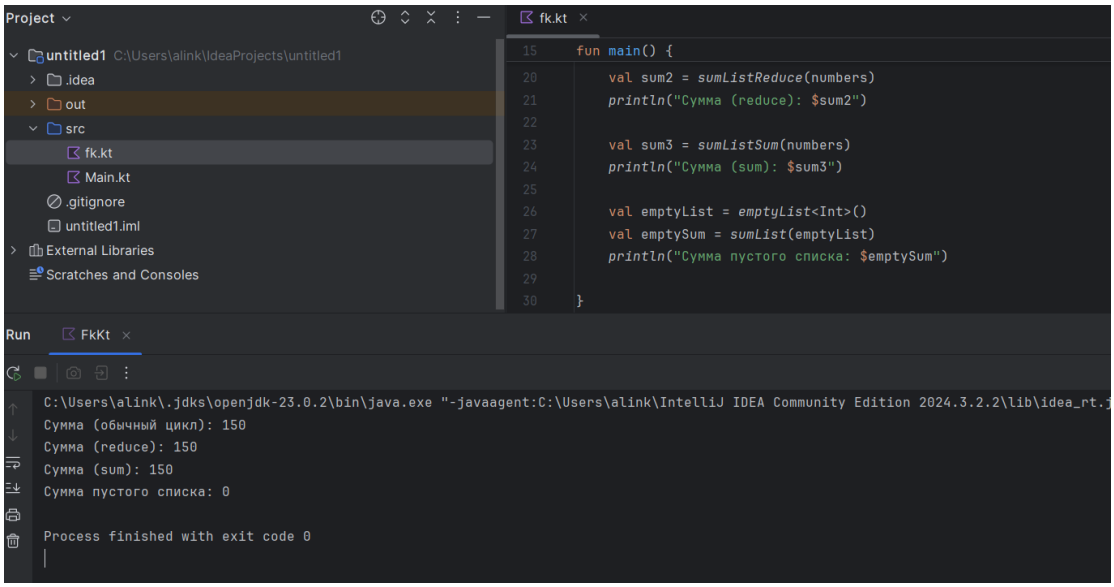
Практическая работа № 8

Выполнили: Андрухова и Загородняя.

1.
```
fun sumList(list: List<Int>): Int {
    var sum = 0
    for (element in list) {
        sum += element
    }
    return sum
}

fun sumListReduce(list: List<Int>): Int {
    return list.reduce { acc, i -> acc + i } // acc - аккумулятор, i - текущий элемент
}

fun sumListSum(list: List<Int>): Int = list.sum()

fun main() {
    val numbers = listOf(10, 20, 30, 40, 50)
    val sum1 = sumList(numbers)
    println("Сумма (обычный цикл): $sum1")

    val sum2 = sumListReduce(numbers)
    println("Сумма (reduce): $sum2")

    val sum3 = sumListSum(numbers)
    println("Сумма (sum): $sum3")

    val emptyList = emptyList<Int>()
```

```
    val emptySum = sumList(emptyList)
    println("Сумма пустого списка: $emptySum")

}
```

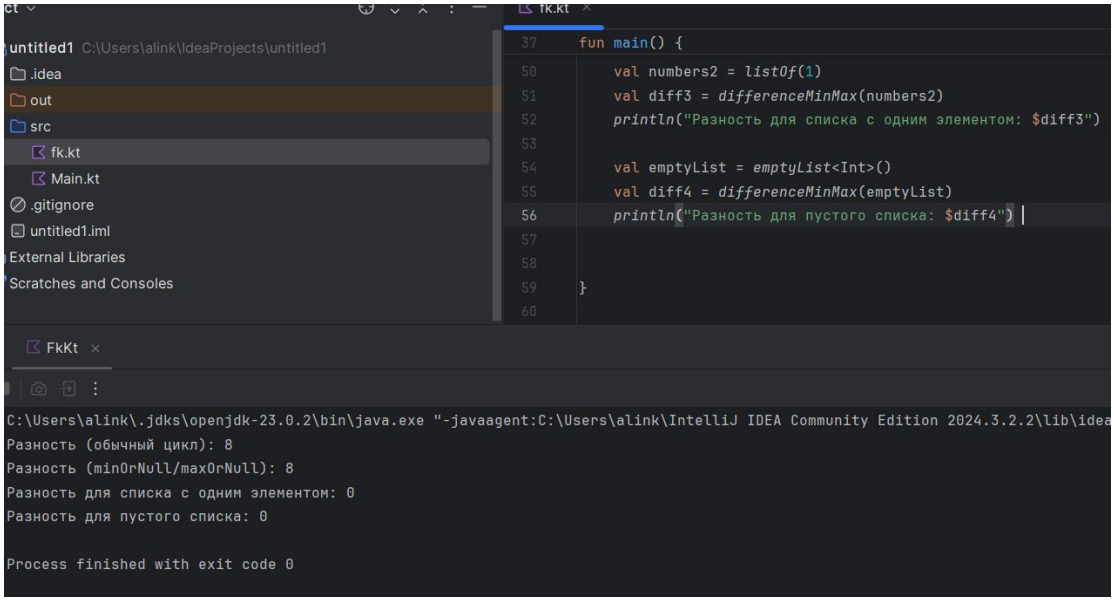


2.import kotlin.math.max
import kotlin.math.min

```
fun differenceMinMax(numbers: List<Int>): Int {
    if (numbers.isEmpty()) {
        return 0
    }
    var minVal = numbers[0]
    var maxVal = numbers[0]


    for (number in numbers) {
        minVal = min(minVal, number)
        maxVal = max(maxVal, number)
```

```
    }


    return maxVal - minVal

}

fun differenceMinMaxOrNull(numbers: List<Int>): Int {
    val min = numbers.minOrNull()
    val max = numbers.maxOrNull()

    if (min == null || max == null) {
        return 0
    }

    return max - min
}
fun main() {
    val numbers1 = listOf(5, 2, 9, 1, 5, 6)
    val diff1 = differenceMinMax(numbers1)
    println("Разность (обычный цикл): $diff1")


    val diff2 = differenceMinMaxOrNull(numbers1)
    println("Разность (minOrNull/maxOrNull): $diff2")

    val numbers2 = listOf(1)
    val diff3 = differenceMinMax(numbers2)
```

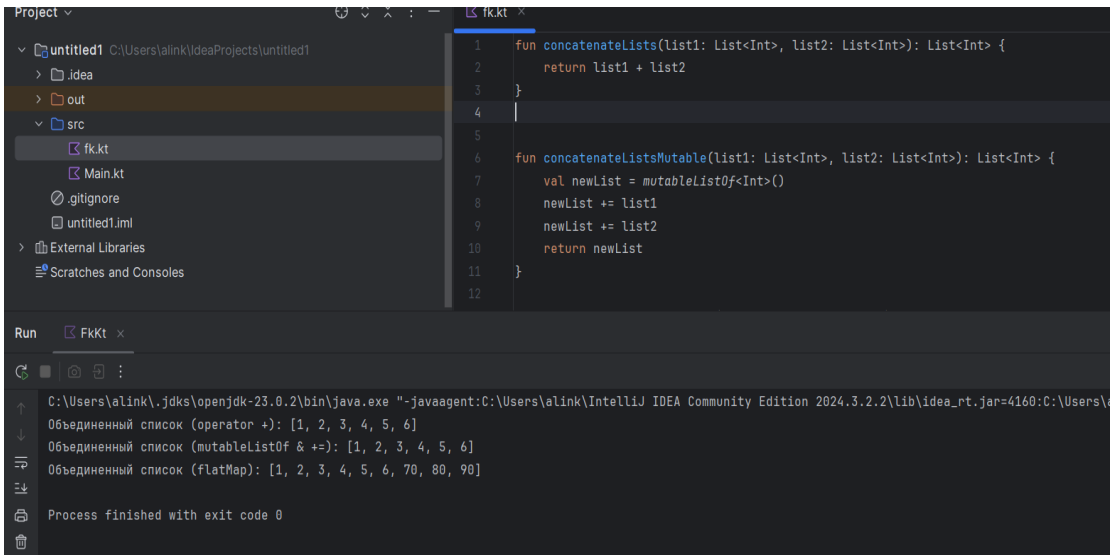
```
    println("Разность для списка с одним элементом: $diff3") // Вывод:
0
    val emptyList = emptyList<Int>()
    val diff4 = differenceMinMax(emptyList)
    println("Разность для пустого списка: $diff4")
}
```




```
3.fun concatenateLists(list1: List<Int>, list2: List<Int>): List<Int> {
    return list1 + list2
}
```

```
fun concatenateListsMutable(list1: List<Int>, list2: List<Int>): List<Int>
{
    val newList = mutableListOf<Int>()
    newList += list1
    newList += list2
    return newList
}
```

```
fun concatenateMultipleLists(vararg lists: List<Int>): List<Int> =
lists.flatMap { it }

fun main() {

    val list1 = listOf(1, 2, 3)
    val list2 = listOf(4, 5, 6)

    val combinedList = concatenateLists(list1, list2)
    println("Объединенный список (operator +): $combinedList")

    val combinedListMutable = concatenateListsMutable(list1, list2)
    println("Объединенный список (mutableListOf & +=):
$combinedListMutable")

    val list3 = listOf(70, 80, 90)
    val multipleCombined = concatenateMultipleLists(list1, list2, list3)
    println("Объединенный список (flatMap): $multipleCombined")

}
```

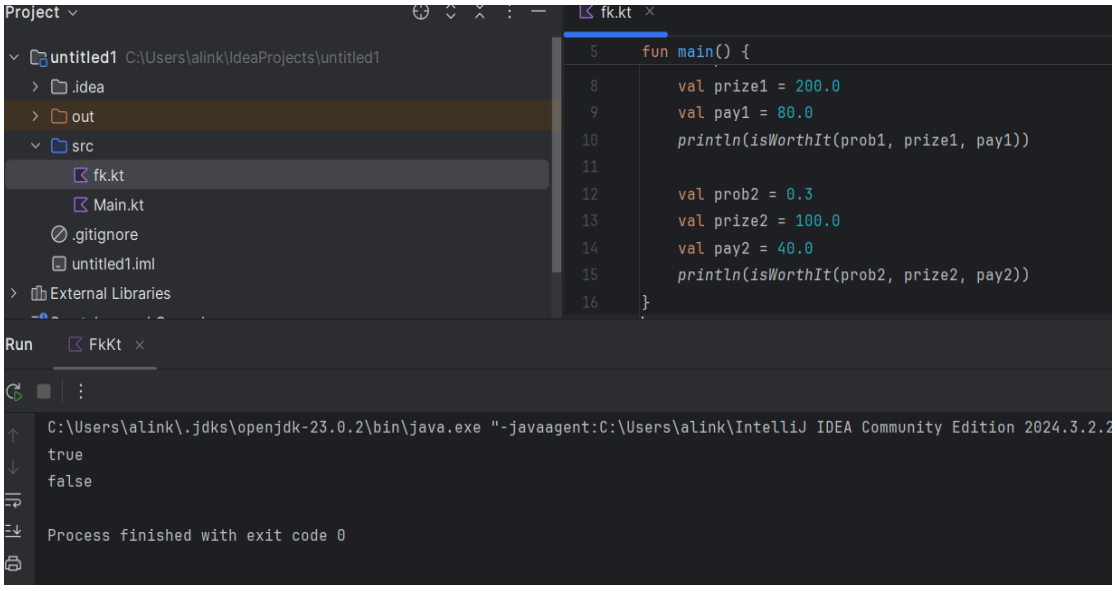


4.fun isWorthIt(prob: Double, prize: Double, pay: Double): Boolean {
    return prob * prize > pay
}

fun main() {

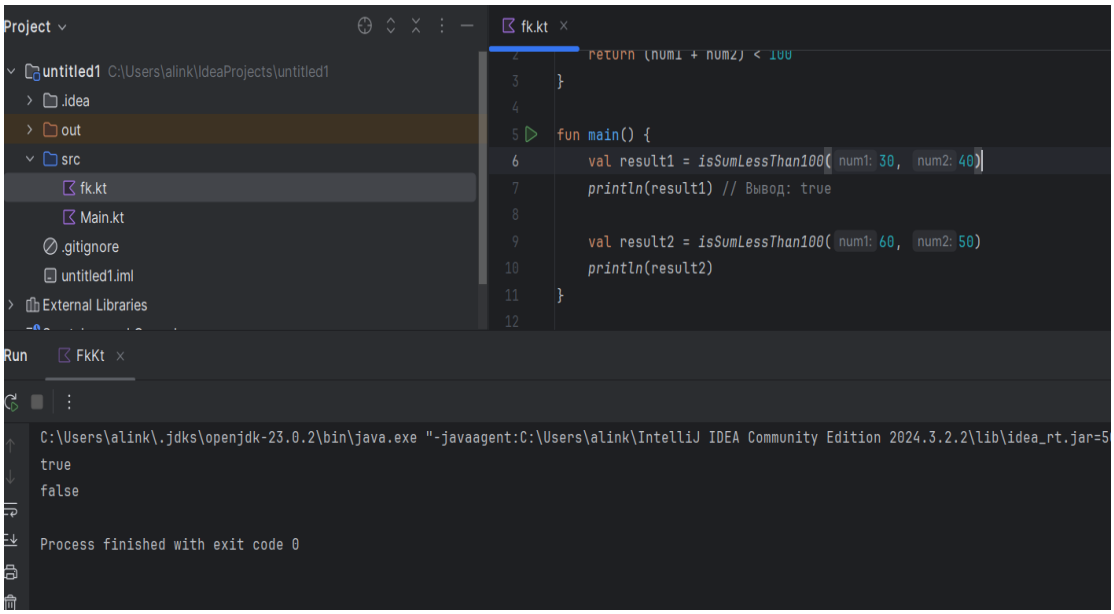
    val prob1 = 0.5
    val prize1 = 200.0
    val pay1 = 80.0
    println(isWorthIt(prob1, prize1, pay1))

    val prob2 = 0.3
    val prize2 = 100.0
    val pay2 = 40.0
    println(isWorthIt(prob2, prize2, pay2))
}



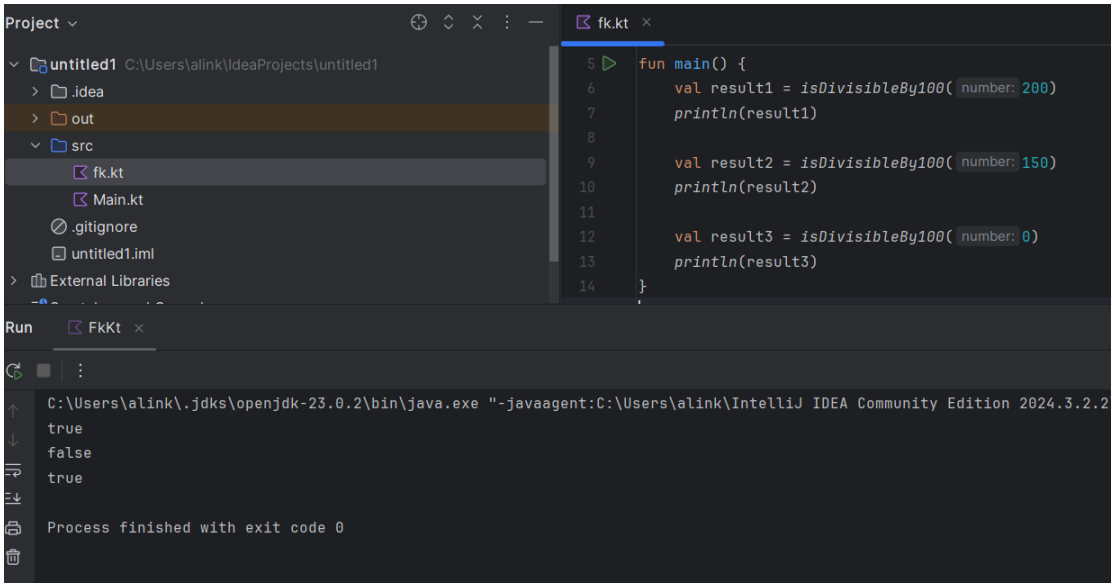
6.fun isSumLessThan100(num1: Int, num2: Int): Boolean {
    return (num1 + num2) < 100
}

```
fun main() {
    val result1 = isSumLessThan100(30, 40)
    println(result1) // Вывод: true

    val result2 = isSumLessThan100(60, 50)
    println(result2)
}
```

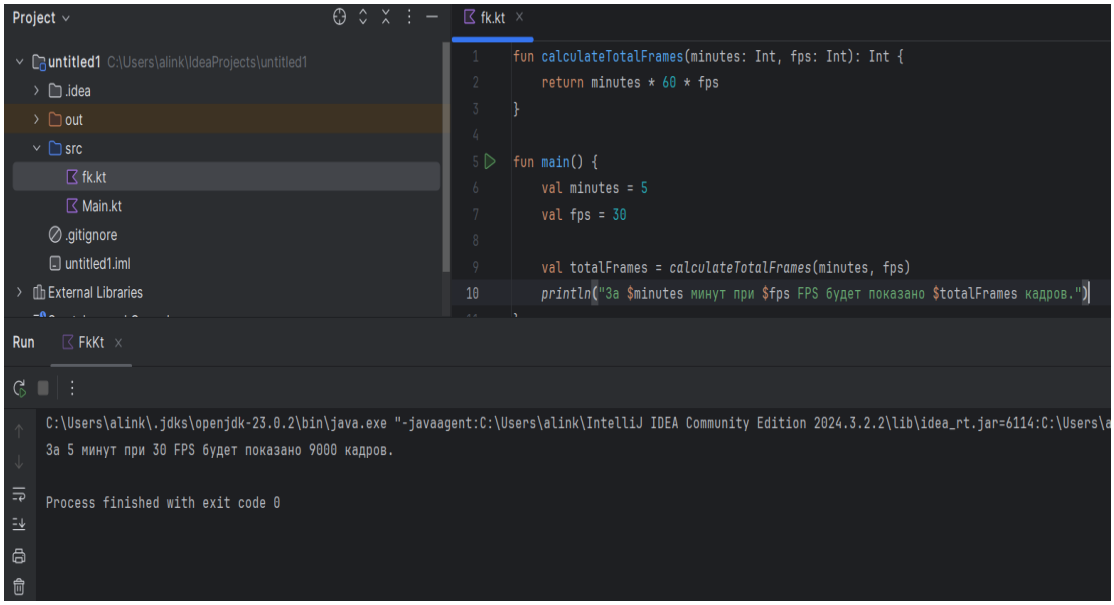



```
7.fun isDivisibleBy100(number: Int): Boolean {
    return number % 100 == 0
}

fun main() {
    val result1 = isDivisibleBy100(200)
    println(result1)

    val result2 = isDivisibleBy100(150)
    println(result2)
```

```
    val result3 = isDivisibleBy100(0)
    println(result3)
}
```

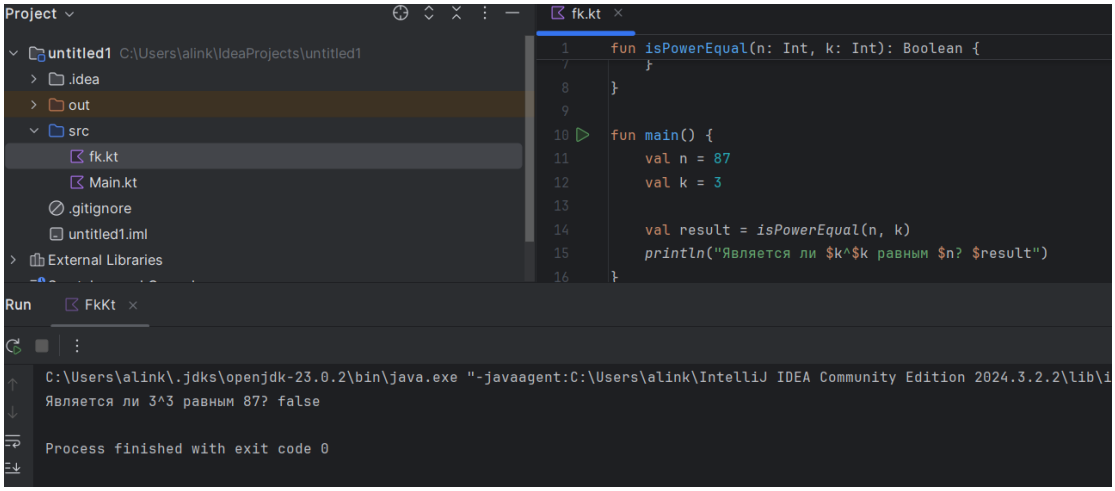



```
8.fun calculateTotalFrames(minutes: Int, fps: Int): Int {
    return minutes * 60 * fps
}

fun main() {
    val minutes = 5
    val fps = 30

    val totalFrames = calculateTotalFrames(minutes, fps)
    println("За $minutes минут при $fps FPS будет показано
$totalFrames кадров.")
}
```



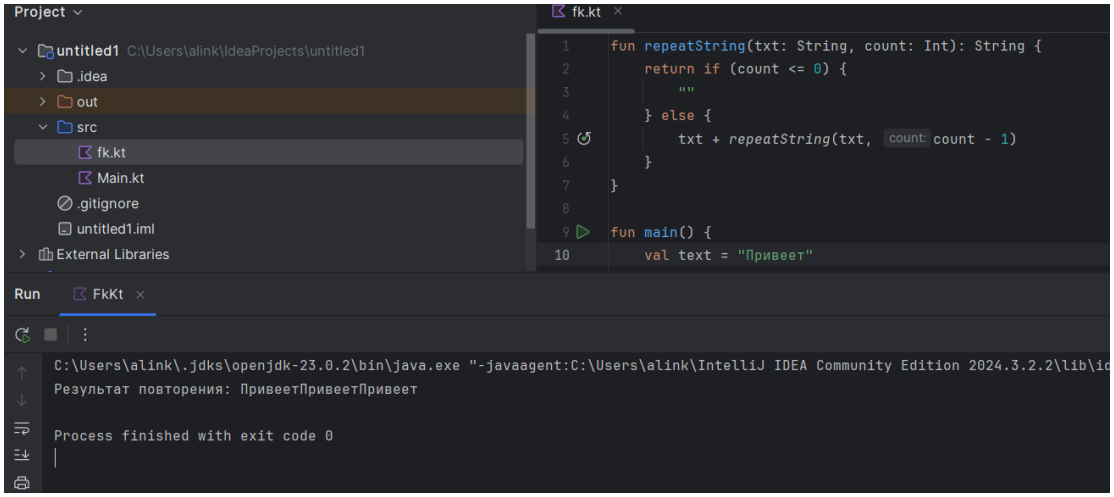
9.fun isPowerEqual(n: Int, k: Int): Boolean {

```
    return if (k <= 0) {
        false
    } else {
        val power = Math.pow(k.toDouble(), k.toDouble()).toInt()
        power == n
    }
}

fun main() {
    val n = 87
    val k = 3

    val result = isPowerEqual(n, k)
    println("Является ли $k^$k равным $n? $result")
}
```



10.fun repeatString(txt: String, count: Int): String {

```
    return if (count <= 0) {
        ""
    } else {
        txt + repeatString(txt, count - 1)
    }
}

fun main() {
    val text = "Привеет"
    val repetitions = 3

    val repeatedText = repeatString(text, repetitions)
    println("Результат повторения: $repeatedText")
}
```



12.

```kotlin
fun google(number: Int): String {
    val prefix = "G"
    val suffix = "gle"
    val oCount = number

    val oString = "o".repeat(oCount)

    return prefix + oString + suffix
}


fun main() {
    println(google(5))
    println(google(0))
    println(google(2))
    println(google(1))
    println(google(3))

}
```

13.

```kotlin
fun greetWorld() {
    println("Привет, мир!")
}

fun main() {
    greetWorld()
}
```

14.

```kotlin
fun sum(a: Int, b: Int): Int {
    return a + b
}

fun sumShort(a: Int, b: Int): Int = a + b

fun main() {
    val num1 = 5
    val num2 = 10
    val result = sum(num1, num2)
    println("Сумма чисел $num1 и $num2 равна $result")
```

```
    val resultShort = sumShort(8, 2)
    println("Сумма чисел 8 и 2 равна $resultShort")
}
```

15.

```
fun найтиБольшееЧисло(число1: Int, число2: Int): Int {
    return if (число1 > число2) {
        число1
    } else {
        число2
    }
}

fun найтиБольшееЧислоКратко(число1: Int, число2: Int): Int =
maxOf(число1, число2)

fun main() {
    val первоеЧисло = 10
    val второеЧисло = 5

    val большееЧисло = найтиБольшееЧисло(первоеЧисло, второеЧисло)
    println("Большее число между $первоеЧисло и $второеЧисло:
$большееЧисло")

    val большееЧислоКратко = найтиБольшееЧислоКратко(первоеЧисло,
второеЧисло)
    println("Большее число между $первоеЧисло и $второеЧисло (краткая
версия): $большееЧислоКратко")

    val число3 = -3
    val число4 = 7

    val большееЧисло2 = найтиБольшееЧисло(число3, число4)
    println("Большее число между $число3 и $число4: $большееЧисло2")
}
```

16.

```
fun isEven(number: Int): Boolean {
    return number % 2 == 0
}

fun main() {
    println("10 is even: ${isEven(10)}")
    println("7 is even: ${isEven(8)}")
    println("0 is even: ${isEven(0)}")
    println("-4 is even: ${isEven(-3)}")
    println("-5 is even: ${isEven(-5)}")
}
```

17.

```
fun factorial(n: Int): Long {
    if (n < 0) {
        throw IllegalArgumentException("Факториал не определен для
отрицательных чисел")
    }
    return when (n) {
        0 -> 1
        1 -> 1
```

```
        else -> n.toLong() * factorial(n - 1)
    }
}

fun main() {
    val number = 5
    try {
        val result = factorial(number)
        println("Факториал числа $number равен $result")
    } catch (e: IllegalArgumentException) {
        println(e.message)
    }

    val negativeNumber = -3
    try {
        val result = factorial(negativeNumber)
        println("Факториал числа $negativeNumber равен $result")
    } catch (e: IllegalArgumentException) {
        println(e.message)
    }

    val zero = 0
    try {
        val result = factorial(zero)
        println("Факториал числа $zero равен $result")
    } catch (e: IllegalArgumentException) {
        println(e.message)
    }
}
```

20.

```
fun findMax(arr: IntArray): Int {
    if (arr.isEmpty()) {
        throw IllegalArgumentException("Array cannot be empty")
    }

    var max = arr[0]

    for (num in arr) {
        if (num > max) {
            max = num
        }
    }

    return max
}

fun main() {
    val arr1 = intArrayOf(1, 5, 2, 8, 3)
    println(findMax(arr1))

    val arr2 = intArrayOf(-1, -5, -2, -8, -3)
    println(findMax(arr2))

    val arr3 = intArrayOf(10)
    println(findMax(arr3))

    val emptyArr = intArrayOf()
    try {
```

```
        println(findMax(emptyArr))
    } catch (e: IllegalArgumentException) {
        println(e.message)
    }

}
```

21.

```
fun sortArray(arr: IntArray): IntArray {

    val sortedArr = arr.copyOf()

    sortedArr.sort()

    return sortedArr
}

fun main() {
    val arr1 = intArrayOf(5, 2, 8, 1, 9, 3)
    val sortedArr1 = sortArray(arr1)
    println(sortedArr1.contentToString())

    val arr2 = intArrayOf()
    val sortedArr2 = sortArray(arr2)
    println(sortedArr2.contentToString())

    val arr3 = intArrayOf(5)
    val sortedArr3 = sortArray(arr3)
    println(sortedArr3.contentToString())

}
```

22.

```
fun isPalindrome(text: String): Boolean {
    val cleanedText = text.lowercase()
    return cleanedText == cleanedText.reversed()
}
fun main() {
    val exampleText = "Anna"
    println("Является палиндромом: ${isPalindrome(exampleText)}")

    val anotherText = "Kotlin"
    println("Является палиндромом: ${isPalindrome(anotherText)}")
}
```

23.

```
fun countCharacters(text: String): Int {
    return text.length
}

fun main() {
    val exampleText = "Привет"
    println("Количество символов: ${countCharacters(exampleText)}")
}
```

24.

```
fun convertToUpperCase(text: String): String {
    return text.uppercase()
}

fun main() {
```

```
    val exampleText = "Как дела?"
    println(convertToUpperCase(exampleText))
}
```